

Glassbox Debugging: Code as Interface for No-Code

Niv Benaloul

Open University of Israel
Dept. of Mathematics and Computer Science
Ra'anana 4353701, Israel
beniv32@post.openu.ac.il

David H. Lorenz[✉]

Open University of Israel
Dept. of Mathematics and Computer Science
Ra'anana 4353701, Israel
lorenz@openu.ac.il

Abstract

No-code platforms and source-level debugging are fundamentally at odds over the role of code in software development. This tension arises because no-code platforms abstract away from code to prioritize simplicity and ease of use, while source-level debugging depends on visibility into the code that actually executes. Even when no-code platforms do generate code explicitly, they often conceal it from developers, treating it as a blackbox. This creates a debugging barrier: users can observe behavior in the visual interface, but cannot trace that behavior to the code that produced it.

To overcome this barrier, this paper introduces a glassbox approach to no-code debugging. In this approach, the generated code behind visual components is exposed as a read-only, synchronized debugging view. The visual program remains the source-of-truth, while the generated code becomes an explanatory interface for inspecting behavior and synchronizing control. The approach further supports cross-target and cross-session debugging, making generated artifacts, runtime behavior, breakpoints, logs, and execution snapshots comparable across target languages and debugging sessions.

By adding source-level inspection and synchronized control without requiring users to edit generated code, glassbox debugging may extend no-code platforms toward fuller development environments, while preserving their accessibility for citizen developers and transitional users. We instantiate this approach in GLANCER, a prototype glassbox debugger for BLOCKLY. Because BLOCKLY can generate multiple target languages from the same visual program, GLANCER provides a concrete setting for exposing behavioral divergences that single-target blackbox debuggers may hide.

CCS Concepts: • Software and its engineering → Software testing and debugging; Visual languages.

Keywords: Block-based Programming, BLOCKLY, Citizen Developers, Debugging, No-Code Software Development



This work is licensed under a Creative Commons Attribution 4.0 International License.

Onward! '26, Oakland, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2934-8/2026/10

<https://doi.org/10.1145/3840586.3843207>

ACM Reference Format:

Niv Benaloul and David H. Lorenz. 2026. Glassbox Debugging: Code as Interface for No-Code. In *Proceedings of the 2026 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! '26)*, October 4–9, 2026, Oakland, CA, USA. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3840586.3843207>

1 Introduction

No-code development platforms enable citizen developers, who lack traditional programming expertise, to build software applications by eliminating the need for direct coding [4]. To run the application, most of these platforms employ code generation techniques, producing *source code* in a conventional high-level language. Yet, they do not support *source-level debugging*, because they abstract the generated code from developers, essentially treating it as a *blackbox* (Fig. 1).

This blackbox approach to no-code debugging offers a couple of advantages. First, it conceals the complexities of the underlying code. Second, it remains agnostic to the choice of target language, allowing a single visual program to be integrated with different runtime environments. For example, BLOCKLY [14],¹ an established, open-source block-based visual no-code programming environment [36], offers an out-of-the-box choice of five alternative target languages for source code generation: JavaScript [12], Python [30], PHP [27], Lua [21], and Dart [5]. We center this work on BLOCKLY precisely for this reason, its support for generating multiple target languages from a single visual program is what makes the cross-target semantic divergences central to our approach observable.

Shielding users of visual components from the underlying implementation, however, comes at the cost of making debugging significantly more difficult. For instance, despite BLOCKLY's success in lowering the entry barrier for software development, its effectiveness is limited by the lack of even rudimentary online debugging capabilities (such as breakpoint-based debugging [33]).

Being oblivious to the target language poses additional challenges for debugging, as the presence of bugs may depend on the particular language. For example, the *Block-level*

¹BLOCKLY may be described as a no-code or low-code environment, depending on the degree of customization and programming knowledge assumed. However, the debugging gap studied here arises in either case.

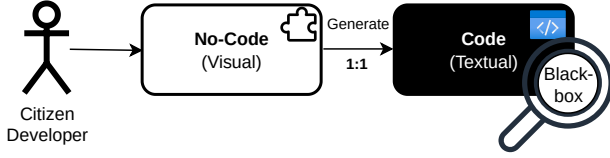


Figure 1. Blackbox approach to no-code debugging.



Figure 2. Screenshot of a simple BLOCKLY program.

Table 1. Execution results for the example shown in Fig. 2.

Target Language	Execution Result
JavaScript	undefined
Python	'#33ff33' (green)
PHP	Warning: Undefined array key...
Lua	nil
Dart	unhandled exception: index out of range

Visual Debugger for BLOCKLY (BVD4B) [44] extends BLOCKLY with basic visual breakpoint-based debugging capabilities and with step control into or over visual blocks, but defaults to JavaScript. Consequently, it overlooks bugs that do not affect JavaScript but would surface, e.g., in Python (or vice versa).

1.1 Motivation

Deliberately hiding generated code from BLOCKLY users to maintain simplicity may protect novice users from complexity, but it creates significant limitations in understanding system behavior, in effectively debugging applications, and in transitioning to more advanced development practices.

Fig. 2 depicts a simple BLOCKLY program that creates a list of three elements and then attempts to access the element at index 0, which is outside the valid range. BLOCKLY does not necessarily throw an out-of-bounds exception like traditional programming languages do when an invalid index is accessed. Instead, its behavior depends on the target language, and the five target languages supported by BLOCKLY behave differently in this case (Tab. 1). In JavaScript, Python, and Lua, the invalid access silently produces different results and therefore goes unnoticed. PHP issues a warning, whereas Dart terminates execution with an out-of-range exception.

Listings 1 and 2 show, respectively, the precise JavaScript and Python code generated for the BLOCKLY program in

Listing 1. BLOCKLY generated JavaScript code for Fig. 2.

```
var traffic_light, current_light;
traffic_light = ['#ff0000', '#ffff00', '#33ff33'];
current_light = traffic_light[-1];
window.alert(current_light); // prints 'undefined'
```

Listing 2. BLOCKLY generated Python code for Fig. 2.

```
traffic_light = None
current_light = None
traffic_light = ['#ff0000', '#ffff00', '#33ff33']
current_light = traffic_light[-1]
print(current_light) # prints last item '#33ff33' (green)
```

Fig. 2. Both versions are consistent in interpreting index 0 as meaning position -1. However, the two behave differently. In JavaScript, accessing index -1 evaluates to *undefined*, while in Python, -1 refers to the last element in the list.

It is possible, of course, to handle such invalid index access cases in lists by creating a custom block in BLOCKLY that checks the index before performing any action, ensuring the index falls within a valid range. The block's comprehensibility can also be improved to make its intended use easier to understand [42]. However, for users relying on the standard BLOCKLY block set, the block's unexpected behavior raises the question: how can developers debug such no-code programs without access to the underlying code?

The lack of visibility into generated code also creates barriers for transitional users that possess some programming knowledge but prefer visual development environments for productivity reasons. These users can find themselves constrained by the limited debugging capabilities and inability to understand the underlying implementation details. Note that a visual debugger, such as BVD4B, may reveal to the developer that the index is undefined, but it does not offer insight into the underlying cause or how to correct it.

1.2 Limitations of Blackbox Debugging

The central limitation of blackbox debugging is that the user observes behavior at the visual interface while execution ultimately occurs in generated code. This separation is harmless when the visual and generated behaviors coincide, but it becomes difficult to diagnose when a bug arises from their mismatch. This difficulty affects both novice and experienced citizen developers [9].

Because users cannot see how visual components translate into target code, special cases such as the out-of-bounds access in Fig. 2 stay hidden, hindering program comprehension and performance tuning [23]. Debugging tools that operate only at the visual-component level, such as BVD4B for BLOCKLY [44] and NUZZLEBUG for Scratch [8], cannot

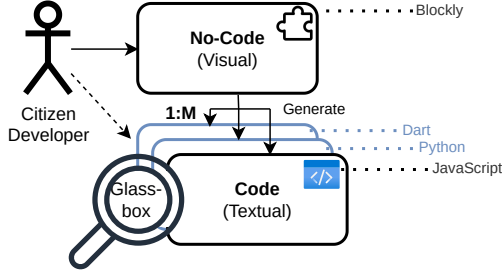


Figure 3. Glassbox approach to no-code debugging.

reach issues rooted in the implementation or in language-specific behavior, as illustrated in Listings 1 and 2. The same opacity blocks a natural progression from visual to textual programming [20] and leaves users unable to verify correctness or maintain cross-target consistency across runtime environments.

1.3 Contribution

This work presents a glassbox debugging approach for BLOCKLY that exposes generated code as a read-only, synchronized debugging interface for no-code programs. The approach maintains bidirectional mappings among visual blocks, generated source lines, runtime state, breakpoints, and execution snapshots across multiple target languages.

The central idea is that generated artifacts should support debugging by acting as explanatory, synchronized, non-authoritative views of the visual program [43]. Unlike blackbox debugging, which reports behavior only through the visual interface, glassbox debugging makes the generated implementation visible and traceable. Unlike whitebox debugging, it does not require users to abandon the visual program or directly edit the generated source. Instead, the visual program remains the source-of-truth, while the generated code becomes an explanatory debugging interface that helps reveal causes hidden by the no-code abstraction.

Concretely, the approach provides three capabilities that together support diagnosis across the visual-textual boundary, while preserving the accessibility of the no-code abstraction. First, synchronized visual-textual debugging lets users inspect generated source code (Fig. 3), understand how visual blocks map to implementation artifacts, and place synchronized breakpoints in either representation before or during execution. Second, cross-target debugging generates and displays multiple target-language implementations of the same visual program, exposing language-specific divergences that single-target debuggers may hide (Figs. 3 and 4). Third, persistent cross-session debugging records execution snapshots, breakpoints, and logs across runs, allowing users to compare debugging evidence across sessions and target languages (Fig. 4).

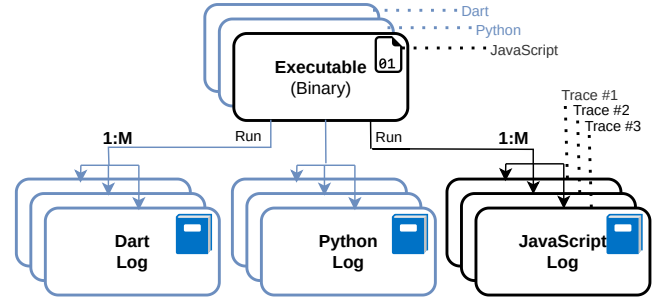


Figure 4. Cross-target, cross-session debugging.

Outline. The paper is organized as follows. Sect. 2 characterizes the visual-textual gap in no-code debugging. Sect. 3 presents the glassbox debugging approach. Sect. 4 introduces GLANCER, a glassbox debugger that realizes this approach as an extension of BLOCKLY. Sect. 5 discusses evaluation. Throughout the paper, *Glassbox* refers to the debugging approach, *GLANCER* to its implementation, and *BLOCKLY* to the target platform used to instantiate and evaluate it.

2 The No-Code Debugging Gap

Debugging no-code programs is difficult not because existing debuggers lack basic debugging functionality, but because they provide it only on one side of the visual-textual boundary. To frame the discussion, we assume that a debugger, independent of platform and approach, is generally expected to support three core capabilities:

- **Execution flow control.** The ability to halt, step through, resume, and otherwise govern the progression of a running program. Granular flow control, including breakpoints, step-into, step-over, and conditional halts, is what turns observation into investigation.
- **State inspection.** The ability to observe the values of variables and the structure of runtime state at chosen points in execution. Effective state inspection requires deciding on both the variables surfaced and the level of detail, since exposing too little obscures the cause of a bug while exposing too much overwhelms the user.
- **Code inspection.** The ability to examine the program components that are being executed, be it visual blocks or textual code. Without code inspection, observed behavior remains an outcome the user can react to but cannot trace back to the program constructs that produced it.

We use three terms for debugging approaches, distinguished by how each treats generated code relative to the visual abstraction. A *blackbox* debugger stays at the visual interface and hides the code, like BVD4B and NUZZLEBUG do. A *whitebox* debugger operates on the code and requires leaving

the visual abstraction, like gdb [45] and pdb [41] do. A *glassbox* debugger exposes the code as a read-only, synchronized view while the visual program remains authoritative.

2.1 Visual-Textual Bias

Blackbox and whitebox debuggers occupy opposite ends of the design space. Blackbox debuggers preserve the visual interface but hide the generated implementation, while whitebox debuggers expose the implementation but force users to leave the no-code abstraction. Neither approach supports the mixed visual-textual workspace needed by transitional users, nor do they leverage the one-to-many relationship between a visual program and the target-language artifacts it generates. As a result, current debugging tools provide core debugging capabilities either visually or textually, but only within the visibility constraints of that interface. This visual-textual gap shapes how blackbox and whitebox debugging approaches support the three core debugging capabilities.

On the one hand, blackbox debugging centers on the visual interface of platforms such as BLOCKLY. Debuggers that implement this approach, including BVD4B [44] for BLOCKLY and NUZZLEBUG [8] for Scratch, expose control flow and program state at the granularity of visual blocks while intentionally hiding the generated source code. This design aligns with citizen developers, who often lack traditional programming expertise and benefit from remaining within the no-code abstraction.

On the other hand, whitebox debugging centers on the textual interface of conventional programming languages. Debuggers that implement this approach, including gdb [45], expose source lines, in-scope variables, and runtime execution steps, and allow users to control execution directly at the level of source code. This design reflects the needs of professional software developers, but requires users to leave the visual abstraction.

With respect to execution flow control, during blackbox debugging, breakpoints attach to visual blocks and stepping advances block by block. During whitebox debugging, breakpoints attach to source lines and stepping advances instruction by instruction. Each approach controls execution within its own interface, but neither relates a halt condition in one interface to the corresponding state in the other. The two granularities are also not simple multiples of each other: a single BLOCKLY block often compiles to many lines, and some generated lines are shared by sibling blocks.

With respect to state inspection, during blackbox debugging, only user-defined block variables are presented. BVD4B, for instance, hides the synthetic counters and helper bindings introduced by the generator. During whitebox debugging, the full runtime stack is visible, including temporary variables and library bindings. Each approach exposes the variables that exist within its own interface, but neither exposes how a value in one interface corresponds to a value in the other.

With respect to code inspection, during blackbox debugging, the program is the visual block model, which the user reads and reasons about directly. During whitebox debugging, the program is the textual source, every line of which is open for inspection. Each approach makes the program visible within its own interface, but neither exposes the relationship between the visual and textual interfaces, where the generator’s behavior, and potential bugs, reside.

2.2 Challenges for Glassbox Debugging

The resulting gap motivates a hybrid glassbox approach that synchronizes the visual interface with the generated code. However, bridging the abstraction gap between no-code and code introduces several more design questions:

Which language is being debugged? A blackbox debugger anchored to a single target language hides cross-target divergence. This limitation is particularly important in BLOCKLY, where one visual program can be translated into multiple target programming languages with different runtime behavior. A whitebox debugger, in contrast, forces users to abandon the visual interface and commit to a specific target language, often the very thing the no-code platform was meant to spare them from. Exporting the generated code into a traditional debugger is technically possible, but it undermines the accessibility promise that motivated no-code adoption in the first place.

Why not normalize? A natural counter-question is whether divergence between target languages could be eliminated upstream by routing every generator through compatibility layers that mask language differences and project a single uniform semantics back to the visual interface. This form of normalization is technically possible, but it would obscure the target-language semantic differences that the debugger is intended to expose.

What would normalization erase? A normalized BLOCKLY could commit to one semantic model for every block, regardless of the target language to which the user eventually deploys. Disagreements between targets would then be suppressed by the compatibility layer, and the user would lose the ability to ask which model their generated program actually obeys at runtime. For a transitional user moving between visual and textual programming, that loss is pedagogical: cross-target divergence is the most direct evidence available that the same program can mean different things in different languages, and a normalized abstraction strips it away. Even setting aside pedagogy, the choice between target semantics is in itself a design decision with no canonical answer [23]: should out-of-bounds list access raise an error, return undefined, wrap around, or coerce?

Whose state is inspected? Generated code routinely introduces variables that have no counterpart in the visual program. When such generator-introduced state diverges

Table 2. Comparison of blackbox and whitebox debugging approaches with the new glassbox approach.

Code visibility	Blackbox	Whitebox	Glassbox
User	Citizen Dev.	Soft. Dev.	Transitional Dev.
Abstraction	No-Code	Code	Hybrid
Interface	Visual	Textual	Synchronized

from the user’s intent, a blackbox debugger that surfaces only block variables cannot explain the divergence, while a whitebox debugger has no notion of the visual interface to map the divergence back to.

What persists across sessions and across languages?

Whitebox debuggers routinely capture stack traces and other replayable artifacts. Existing no-code debuggers reset between runs and discard cross-target evidence, even though comparing the same visual program across multiple targets is precisely where language-specific bugs may reveal themselves. This disagreement between targets is an informative debugging signal, but is structurally invisible to current no-code debuggers.

What is the cost of blackbox visibility for the no-code abstraction? Blackbox debugging honors the simplicity promise of no-code platforms but reports symptoms without causes. When a failure emerges from the abstraction gap, the user can see that something went wrong but not why it went wrong. For citizen developers, this is a learning bottleneck, since the failure may be the first time they encounter a particular programming concept or language-specific behavior. Their only available option is an abrupt transition to a whitebox tool that assumes language fluency they do not yet possess. The literature on no-code platforms acknowledges such transitional users explicitly, including learners using visual programming as a stepping stone toward textual programming [20] and code-fluent professionals using no-code tools for rapid prototyping [3], enterprise productivity [4], and competitive advantage [22].

3 Glassbox Debugging

We present a glassbox approach to BLOCKLY debugging that synchronizes visual blocks, generated source code, runtime state, and debugging metadata during online debug sessions. The approach occupies the mixed visual–textual space between blackbox and whitebox debugging, supporting execution flow control, state inspection, and code inspection across interfaces rather than within a single interface (Tab. 2). It further extends this synchronization across target languages, making generated artifacts and debugging evidence comparable across the one-to-many relationship between a visual program and its implementations.

3.1 Design Principles

The following design principles shape the glassbox approach.

Semantic variability as diagnostic signal. Semantic variability is treated as debugging evidence rather than as a defect to be normalized away. This design choice follows from the fact that different domains make different semantic choices [49], and that BLOCKLY’s open-source extensibility [36] expressly invites domain-specific generators that may diverge.

Safe code inspection. The textual interface is exposed but immutable (i.e., readable but not editable). Users see exactly the code the generator produced, with all its language-specific quirks, but cannot drift the visual model out of sync by hand-editing the text. The visual interface remains the source-of-truth, while the generated code becomes a responsive view that explains the *why* behind runtime behavior.

Synchronized control flow. Standard whitebox debugging controls, such as stepping and breakpoints, are exposed at both interfaces, with bidirectional synchronization between them. This design choice entails two additional requirements.

- **Breakpoint correspondence** Breakpoints must have a single synchronized meaning across blocks and generated source lines, even when one BLOCKLY block compiles to many source lines or one source line is shared by sibling blocks.
- **Granularity-aware stepping** Stepping must support both visual and textual granularities, because a single visual step may correspond to many code-interface steps, and vice versa.

State inspection across interfaces. State inspection spans the visual program, the generated code, and the underlying runtime without collapsing them into a single view. This design choice entails two additional requirements that have no direct counterpart in either purely blackbox or purely whitebox tools.

- **Perspective-aware state.** User-defined block variables, generator-introduced bindings, and the underlying runtime stack each reveal a different slice of a failure. The glassbox debugger lets the user move between these perspectives without overwhelming the visual presentation.
- **Cross-session persistence.** Debugging evidence, execution traces, breakpoints, and snapshots are preserved across runs so that transient debugging accumulates into shareable state, enabling comparison between configurations, revisions, and target languages over time.

The abstraction gap as a debugging asset. BLOCKLY’s ability to emit code in multiple target languages is normally

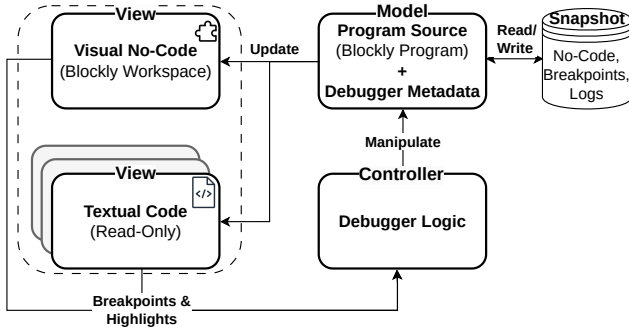


Figure 5. Architecture.

treated as a deployment convenience. It could also be viewed as a liability (Sect. 1). However, the glassbox approach treats it as a debugging asset, since comparing executions across targets reveals language-specific bugs that no single-target debugger can surface (Tab. 1). This raises further tradeoffs.

- **Multiple target languages.** Maintaining a cross-target execution view increases generator and runtime overhead, but multiplies the debugger’s diagnostic reach by exposing where target semantics disagree.
- **Multiple generators per language.** Even within a single target language, alternate generators (e.g., performance-oriented vs. readability-oriented) may produce divergent behavior for the same visual program. The glassbox debugger must remain neutral to generator choice and treat the generator as part of the debuggable surface.

The remaining subsections describe how the glassbox debugger realizes these design principles.

3.2 Multiple Concurrent Views

Conceptually, the debugging workflow follows the *Model-View-Controller (MVC)* architectural pattern [26], as depicted in Fig. 5. The graphical user interface comprises multiple *views*, including the BLOCKLY workbench decorated with runtime breakpoints and program-state inspection, several read-only code editors for the various target programming languages, each providing mechanisms for manipulating breakpoints and controlling program execution, and dedicated panels for viewing program snapshots and execution logs.

The user interacts with a view by setting breakpoints or stepping execution, either in the BLOCKLY workspace or in one of the code editors. The controller processes the action and updates the model, which in turn propagates the change back to all views, keeping breakpoints and highlights synchronized across visual and textual representations.

Collectively, these views expose the relationship between visual blocks and generated code as a navigable interactive interface. Three capabilities follow.

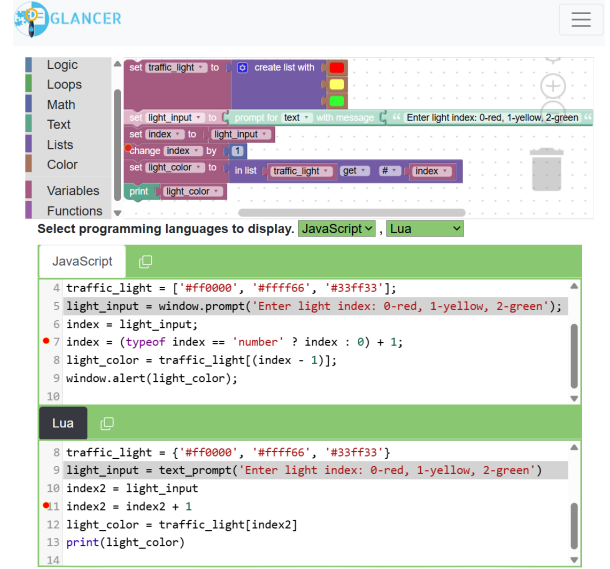


Figure 6. A BLOCKLY workspace, a JavaScript code editor, and a Lua code editor in GLANCER, all displaying synchronized highlights and breakpoints.

Awareness. Source-level visibility exposes how a block is generated and executed in the target language, including generator-introduced constructs that no purely visual view would reveal.

Interoperability. Synchronizing the visual and textual interfaces supports navigation between them.

For example, Fig. 6 shows the traffic-light program in BLOCKLY together with two code editors, one for JavaScript and one for Lua. Highlighting line 5 of the JavaScript code also highlights the corresponding ancestor block ‘set light_input’ and its horizontally stacked siblings, as well as line 9 of the Lua code. Multiple breakpoints, marked with a red dot, are set on certain blocks and are kept in sync with code-line breakpoints in both JavaScript (line 7) and Lua (line 11).

Portability. When the source code is accessible, the developer can carry it, together with metadata such as comments and breakpoints, into a traditional *Integrated Development Environment (IDE)*. This supports diagnosis by enabling a graceful transition to textual tools and debuggers when in-depth investigation is required.

3.3 Cross-Target Code Generation

BLOCKLY already generates code in five target languages from one visual program. Our glassbox views generate code in all five languages simultaneously and treat the generators as part of the debuggable surface [46].

Cross-target glassbox. Displaying generated code in several languages side by side surfaces implementation differences that would otherwise be invisible to the developer. For example, Fig. 6 shows that the JavaScript generator emits an *explicit* type conversion at line 7,

```
7 index = (typeof index == 'number' ? index : 0) + 1;
```

whereas the Lua generator relies on an *implicit* conversion at line 11,

```
11 index2 = index2 + 1
```

yielding different runtime outcomes.

Cross-target debugging. Because every generated source has the same single BLOCKLY program as its origin, breakpoints set at any abstraction level map cleanly to any other target language, supporting quick cross-target comparisons during debugging.

3.4 Cross-Session Debugging Persistence

Our approach captures each run as a *snapshot* of the BLOCKLY program together with its breakpoints and execution logs.

Collaborative debugging. Version Control Systems (VCSs), such as Git [29], are routine in textual development but less established in block-based programming environments. Some environments integrate VCS support [34] and collaborative workspaces [48] for BLOCKLY. However, the management of debugging metadata in these environments remains largely unaddressed. The snapshot mechanism narrows that gap by preserving enough debugging state to support returning to a previous debug session, comparison across runs, and sharing debugging context with collaborators.

4 GLANCER: A Glassbox No-Code Debugger

GLANCER is a proof-of-concept glassbox debugger implemented as an open-source extension of BLOCKLY [14] and BVD4B [44]. BLOCKLY provides the visual workspace and target-language code generators. BVD4B provides the JavaScript block-level stepping mechanism, which GLANCER extends with synchronized generated-code views, cross-target execution, breakpoint export, and cross-session persistence.

GLANCER’s implementation centers on a shared block-to-code mapping. This mapping connects BLOCKLY block identifiers to generated source locations for each selected target language, and it is used to synchronize highlights, breakpoints, execution state, snapshots, and exported debugging metadata. The rest of this section describes the user interface, the mapping algorithm, synchronization behavior, cross-target execution, and persistence mechanisms.

4.1 User Interface

The debugger is a split view with three regions (Fig. 7).

- **Debugger controls.** Step controls, breakpoint export, snapshots, execution logs, and execution in multiple target languages.
- **Visual workspace.** The BLOCKLY workbench for constructing the program.
- **Glassbox views.** CODEMIRROR [19] editors that display the generated source in user-selected target languages. The views are deliberately *read-only*: editing the generated text would desynchronize it from the visual program, so all program changes are made in blocks while the code views remain a faithful, inspectable projection.

The screenshot in Fig. 7 shows an in-progress debug session. Breakpoints are colored gray when disabled, red when enabled, and red-with-yellow-center when reached. We extend CODEMIRROR with custom line gutters so that breakpoints and highlights can be manipulated directly on the read-only code, while still benefiting from CODEMIRROR’s syntax highlighting and editor features.

4.2 Block-to-Code Mapping

BLOCKLY programs are built by stacking blocks vertically using *previous* and *next* statement connectors, with three kinds of inputs nested inside each statement block: *value inputs* (horizontal sockets carrying value-producing blocks, translated inline into the generated code), *dummy inputs* (horizontal slots holding static fields, labels, or non-interactive elements that translate via templates), and *statement inputs* (vertical containers for nested loop or conditional bodies that translate into a nested code scope).

This input taxonomy yields a hierarchical structure with a clear correspondence between a block group and its generated code, which GLANCER uses for synchronization. For each target language L , a map $M_L : b \mapsto \{l_1, \dots, l_k\}$ is maintained, where b is a BLOCKLY block identifier and each l_i is a contiguous range of generated source lines linked to b . When a nested value block contributes to a source line generated by an enclosing statement block, GLANCER maps the line to the greatest horizontal ancestor rather than to every nested child. This policy gives synchronization a stable source-level unit even when several visual blocks contribute to one generated line.

The mapping algorithm recursively identifies the greatest horizontal ancestor of each block within its statement scope, handles special cases such as conditional statements and functions, and associates that ancestor’s identifier with the corresponding generated source lines. A workspace event listener rebuilds or updates the mapping as the user edits the program. This map is the glassbox counterpart of the debug symbols used for compiled binaries. GLANCER produces it as a by-product of BLOCKLY’s code generation, so debugging does not require recompilation or a separate symbol-extraction step.

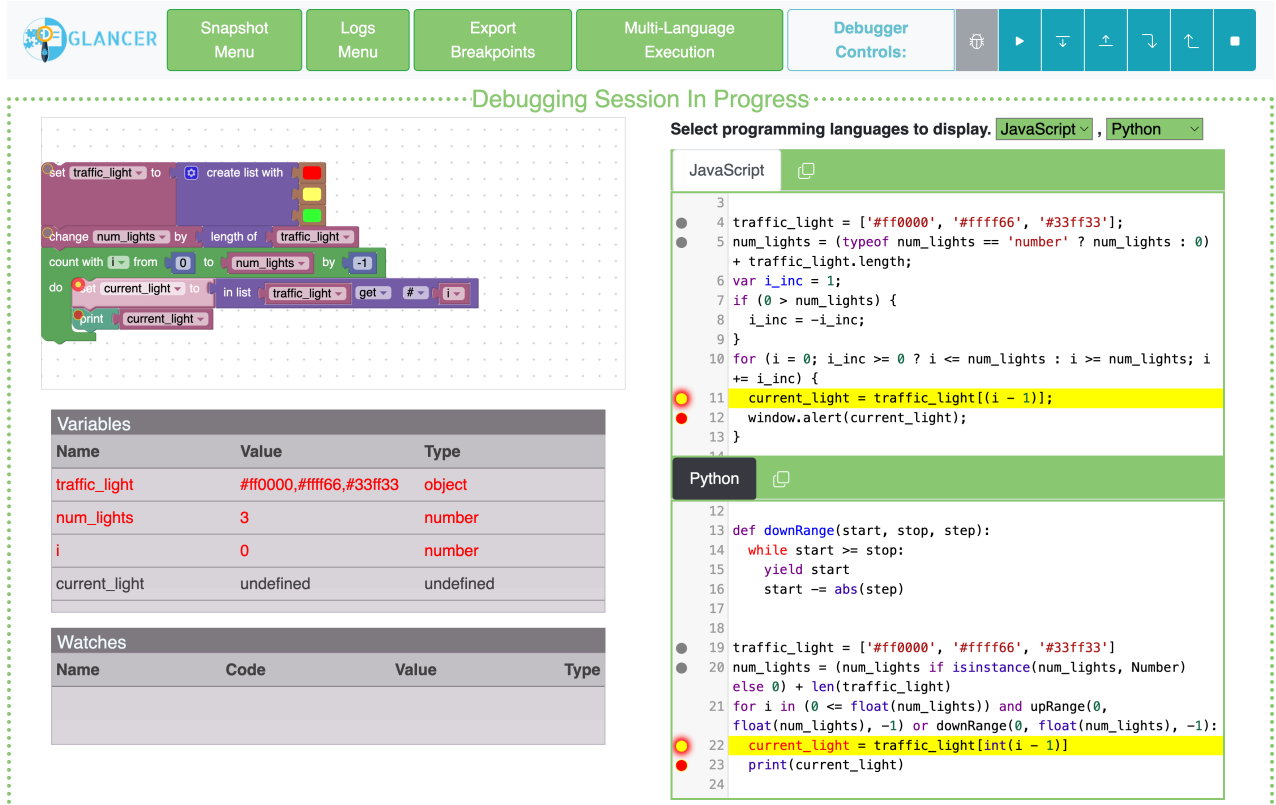


Figure 7. A debugging session in GLANCER paused at a breakpoint corresponding to the highlighted block and code (lines 11 and 22). On the left, a BLOCKLY workspace with breakpoint markings (gray for disabled, red for enabled, red with a yellow center for enabled and reached). On the right, two code editors (JavaScript and Python) displaying the matching breakpoints in the code.

Because GLANCER maintains a separate block-to-code map for each target language, the mapping contains $O(n \cdot m)$ entries for n blocks and m target languages. This avoids constructing explicit pairwise correspondences between generated lines in different languages, which would add unnecessary cross-target mapping overhead.

4.3 Highlights

Highlights are the most direct expression of bidirectional synchronization: selecting one element automatically highlights its counterparts across the visual and textual interfaces, in either direction.

- **Block highlight.** Selecting *Highlight* from a block's BLOCKLY context menu emphasizes the block in a brighter color and highlights the generated code lines for its ancestor across all displayed target languages.
- **Code-line highlight.** Middle-clicking a line number in any code editor emphasizes the line and highlights the generating ancestor block together with all of its offspring blocks.

A highlight is cleared either explicitly from the same context menu or implicitly by performing any debugger step.

4.4 Breakpoints

Breakpoints extend bidirectional synchronization to a stateful artifact: they can be set, disabled, or removed from either interface, and the two sides stay consistent under a single resolution policy rather than as independent overlays.

- **Block breakpoint.** Set, disable, or unset via BLOCKLY's context menu. GLANCER synchronizes the block breakpoint with the code lines generated by the block's ancestor.
- **Code-line breakpoint.** Left-clicking a line number cycles through set, disable, and unset, the convention in text-based debuggers. The code-line breakpoint is automatically linked to the generating ancestor block, keeping the visual workspace and every glassbox view in sync.

A horizontal block group generates a single source-level scope, but may host several block-level breakpoints. GLANCER resolves this ambiguity with an *any-and-ancestor-first* policy. If any descendant of an ancestor block has a set breakpoint, the generated scope receives a breakpoint on its first code line. Its enabled/disabled state is the disjunction of the descendants' states. Conversely, code-line actions act on the ancestor group as a whole. Setting a breakpoint on a code

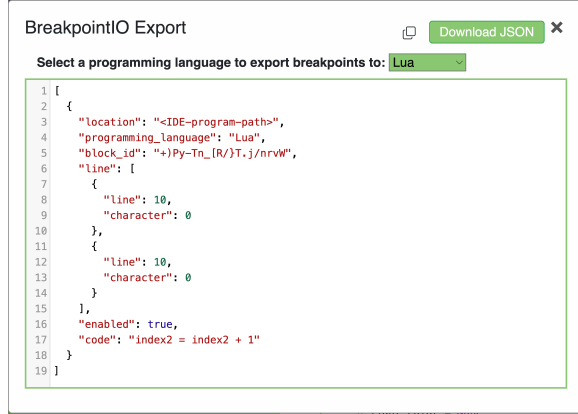


Figure 8. Exported metadata of Lua breakpoints in Fig. 6.

line places one on the generating ancestor block, disabling a code-line breakpoint disables every enabled breakpoint in the group, and removing it clears the entire group. The policy keeps block-level and source-level interactions consistent without requiring the user to track which side is authoritative.

Reach synchronization. During block-level stepping, the current block is highlighted and its breakpoint is marked as reached. The generated code lines for the ancestor block are highlighted in step, and their breakpoint is marked in every displayed target language, letting the user correlate visual structure with runtime behavior across languages.

Breakpoint export. GLANCER exports breakpoint metadata, including line numbers and enabled state, as JSON files following the BREAKPOINTIO [7] convention, which integrates with the VISUAL STUDIO CODE [35] IDE (Fig. 8). The export is derived dynamically from the block-to-code mapping, so any target language supported by BLOCKLY is supported by export. The result is a clean handoff path from GLANCER to a traditional text-based debugger when in-depth investigation is required.

4.5 Cross-Target Execution

GLANCER supports two execution modes over the same authoring artifact. The first, *interactive step-debugging*, drives a single instrumented target: the user sets breakpoints, halts, and steps, while BVD4B inserts asynchronous *Promises* [39] into the generated JavaScript so that execution can pause without altering program logic. The remaining target languages are not independently steppable. Their code views are read-only projections, and any breakpoints or highlights shown there are synchronized markers mapped through the block-to-code mapping. The second mode runs each selected target to completion, feeding the cross-target comparison described next.

	#Run	traffic_light	light_input	index	light_color
1	1	#f0000, #fff66, #33ff33 (object)	-1 (string)	1 (number)	#f0000 (string)
2	2	#f0000, #fff66, #33ff33 (object)	0 (string)	1 (number)	#f0000 (string)
3	3	#f0000, #fff66, #33ff33 (object)	1 (string)	1 (number)	#f0000 (string)
4	4	#f0000, #fff66, #33ff33 (object)	2 (string)	1 (number)	#f0000 (string)

Figure 9. Sample execution logs for the program in Fig. 6.

Cross-target comparison is realized by executing the same BLOCKLY program in any selection of its supported target languages and displaying the outcomes side by side in a comparative table. The outcomes range from successful runs with expected or unexpected outputs to faulty executions, all derived from a single authoring artifact. The view exposes language-specific behavioral divergence directly.

The execution runtime is intentionally treated as a pluggable component of the model, decoupled from the diagnostic interface. Targets can be executed locally via in-browser interpreters, either compiled to WEBASSEMBLY [15] (e.g., Pyodide [40] for Python and php-wasm [37] for PHP) or implemented natively in JavaScript (e.g., Fengari [6] for Lua, alongside the host JavaScript itself).

4.6 Snapshots and Logs

Snapshots. A snapshot stores the BLOCKLY program’s XML representation together with the breakpoint states, in a custom JSON schema. Snapshots are taken *manually* on user request, or *automatically* at the end of each debug session. Loading a snapshot reconstructs the BLOCKLY program, regenerates the source for every selected target language, and restores the breakpoints across all views.

Execution logs. Each debug session contributes a row to the execution log, recording the run number, a timestamp, the final state of BLOCKLY variables, and a link to the corresponding automatic snapshot (Fig. 9). The log is presented through the HANDSONTABLE [17] spreadsheet, with filtering, column hiding, and CSV export, so users can explore execution history with familiar interactions.

Snapshots, execution logs, and breakpoint export together realize cross-session persistence: they carry programs, breakpoints, and execution history forward in time, so diagnosis is not confined to a single run, and provide a clean hand-off path to traditional whitebox debuggers when in-depth investigation is required.

5 Evaluation and Discussion

We evaluate GLANCER as a proof-of-concept system. The evaluation asks two questions: first, whether GLANCER realizes the design goals, and second, whether glassbox debugging exposes diagnostic information that is unavailable in existing blackbox visual debuggers. The evaluation therefore focuses on design-goal coverage and diagnostic observability, rather than end-user debugging performance. Accordingly, it makes

Table 3. Compliance with the debugger design goals. A checkmark indicates prototype-level support. A partial mark indicates support only at the visual-block level.

Capability	Whitebox	Blackbox	Glassbox
Execution Flow Control	✓	✓	✓
State Inspection	✓	✓	✓
Code Inspection	✓	✗	✓
Cross-Target Debugging	✗	✗	✓
Cross-Session Debugging	✓	✗	✓

no claims about user performance. Instead, it asks whether glassbox debugging makes otherwise hidden causes observable. That is, the unit of analysis is diagnostic observability, not human debugging speed or accuracy.

5.1 Design-Goal Compliance

Tab. 3 positions GLANCER against gdb [45] as the whitebox baseline and against BVD4B and NUZZLEBUG [8] as the blackbox state of the art. A capability is marked as supported when the tool exposes it directly in the debugging workflow without requiring manual export to an external environment. A partial mark indicates support only at the visual-block level.

The table reflects the framing of Sect. 2. gdb and GLANCER both expose code at the textual interface. BVD4B and NUZZLEBUG both expose visual blocks but neither maps that exposure to runtime state at the source interface. Only GLANCER occupies all three rows simultaneously, and only GLANCER extends along the cross-target and cross-session axes.

The remaining questions raised in Sect. 2 are answered constructively by the design. *Which language is being debugged* becomes a non-issue as GLANCER treats every selected target as live and surfaces the disagreement between them as a first-class diagnostic signal. *Whose state is inspected* is answered by a unified view that spans block-level user variables and the runtime state of every active target language. *What persists across sessions and languages* is answered by the snapshot and execution-log mechanism (Sect. 4.6). *The cost of blackbox visibility for the no-code abstraction* is illustrated by the cases of Sect. 5.2, which expose concrete bugs that are difficult to surface using a blackbox debugger.

5.2 Diagnostic Coverage

We evaluated diagnostic coverage using a curated suite of eight BLOCKLY programs designed to exercise failures at the visual-textual boundary (Tab. 4). Each program is a representative use case: a small program a developer might plausibly build, the failure they would observe, and what each debugging approach lets them learn about its cause. For the diagnostic-coverage cases, we use BVD4B as the concrete BLOCKLY blackbox baseline. The suite includes cross-target list indexing, implicit type conversion, generator-introduced state, target-specific runtime semantics, undefined variables,

nested-loop counters, and custom generator divergence. For each case, we compared what the blackbox block-level debugger can expose with what GLANCER exposes through synchronized generated code, cross-target execution, runtime logs, and breakpoint mappings. A case is marked as revealed in GLANCER when the relevant cause is directly observable in the debugging interface without manually exporting the program to an external textual debugger. Tab. 4 summarizes the scenarios and the diagnostic signal each one yields under GLANCER compared with the blackbox baseline.

In Fig. 2, a blackbox debugger exposes the visual operation and its resulting value, but it does not show why the same visual block produces different behavior across target languages. GLANCER exposes the generated code and cross-target outputs together, making the target-specific indexing semantics directly observable.

In Fig. 6, a traffic-light program prompts the user for an index and uses it to access a list of colors. Under a blackbox debugger such as BVD4B, the user consistently observes the first color regardless of the input. BVD4B variable inspector confirms that the `index` variable always evaluates to 1 (Fig. 9). This clarifies the symptom but not the root cause, since the generated code that performs the silent coercion is not exposed.

GLANCER exposes this by executing the same program in multiple target languages. The cross-target execution shows that JavaScript, Python, and Dart always print the first color while Lua and PHP behave correctly. Reading the generated source explains the divergence. The JavaScript generator performs an explicit type check and fallback on the index, which silently coerces a string input to 1 and selects the first list element. With both the divergence and its source-level explanation visible, the developer can reason about the correctness implications and choose to enforce input typing or to accept the language-specific behavior knowingly.

In Fig. 7, a BLOCKLY program creates a list of traffic-light colors and attempts to print all its elements. Inconsistent behaviors emerge during the initial list access due to language-specific indexing variations (see also Sect. 1). In a blackbox debugger such as BVD4B, the user can step through the visual program and inspect block-level variables, but the generated code is not exposed. The program exhibits unexpected behavior, and stepping plus block-variable inspection clarify the symptom without surfacing the language-specific cause. In this case, Lua exposes a different failure than that in Fig. 2, an access to an uninitialized global variable:

```
attempt to perform arithmetic on a nil value
(global num_lights).
```

GLANCER reveals the generated source code alongside the visual blocks and enables cross-target execution, which exposes the off-by-one indexing as a correctness-relevant divergence across targets and helps the developer reason about

Table 4. Diagnostic observability of GLANCER across representative BLOCKLY debugging scenarios. A checkmark indicates that the cause of the failure is directly observable. A partial mark indicates that the symptom is visible, but the source-level cause is not.

Failure	Scenario	Blackbox	Glassbox
Indexing semantics	List index 0 (Fig. 2)	✗	✓
Implicit conversion	String index input (Fig. 6)	✗	✓
Runtime binding	Undefined variable (Fig. 7)	✓	✓
Generator-introduced state	Nested loop counter (Fig. 7)	✗	✓
NaN propagation	GCD infinite loop (Fig. 10)	✓	✓
Runtime semantics	Division by zero (Fig. 10)	✗	✓
Numeric semantics	Negative square root	✗	✓
Generator divergence	Custom generator variant	✗	✓

its language-specific cause. The bidirectional highlight mechanism correlates the implicated block with its generated line of code, supporting the diagnosis of where explicit bounds-checking logic is needed.

In Fig. 10, a simple *Greatest Common Divisor (GCD)* algorithm using two inputs `num1` and `num2` contains two subtle bugs. The values are entered as strings (e.g., `'24'`, `'36'`), which become `NaN` once arithmetic is applied, and the algorithm contains faulty logic that ultimately divides by zero.

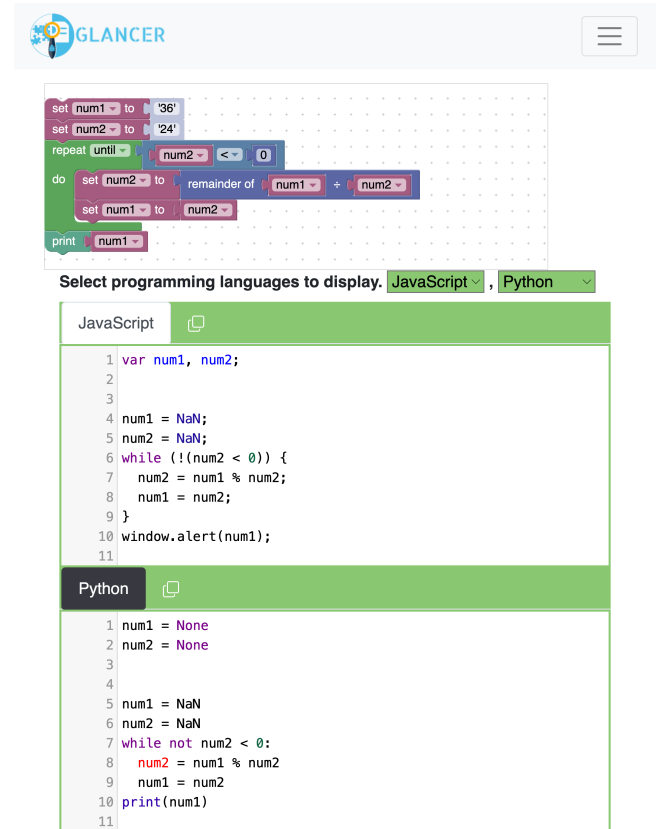
Under unmodified BLOCKLY, the program silently hangs in an infinite loop, offering no clues. BVD4B improves the situation by allowing step-by-step execution: the user can detect the initial `NaN` via state inspection, but after fixing the input type they re-execute and encounter a remaining infinite loop with no further diagnostic signal. GLANCER supports diagnosis directly: viewing the generated code reveals the `NaN` assignment, and cross-target execution surfaces a division-by-zero exception in most of the targets. The combination of visual-code synchronization, runtime logs, and cross-target execution exposes the cause directly in this scenario.

In the negative-square-root case, GLANCER exposes target-dependent numeric behavior that a blackbox debugger hides. In the custom-generator-variant case, GLANCER exposes differences between alternative generated implementations of the same visual block. The full code is omitted due to space constraints.

5.3 Supporting the Transition

Sect. 2 identified the missing *transitional debugger* as the gap between the no-code and code abstractions. GLANCER is one occupant of that gap. Three of its capabilities, taken together, describe a workflow that neither a purely blackbox nor a purely whitebox tool can offer.

Who benefits, and how. GLANCER serves both citizen developers and transitional users, because its capabilities do not all require reading code. This answers a natural objection: if a user can already read the generated JavaScript or Python, why debug in BLOCKLY at all, and if a user cannot, how does exposing the source help rather than add complexity? The

**Figure 10.** A BLOCKLY program with an infinite loop.

two groups rely on different layers. A citizen developer uses the *behavioral* layer: cross-target execution shows that the same program prints different results, or fails in some targets but not others, and bidirectional highlighting points to the responsible block, all without reading a line of syntax. A transitional user, who is acquiring code literacy, also reads the *source* layer, where the generated code explains why the behaviors diverge. For this user, the read-only view is the mechanism of the transition, not a prerequisite for it.

Explanations versus artifacts. Prior work on *grounded abstraction matching* [28] argues that end-user programmers are often better served by explanations expressed in familiar abstractions, such as structured natural language, than by raw generated syntax. We regard such explanations as complementary to, rather than in competition with, glassbox debugging: surfacing block-level or natural-language explanations of a divergence is a natural extension that can be presented as future work. The concrete artifact must nonetheless remain available, because the signal GLANCER exposes, cross-target semantic divergence, is precisely what a single normalized abstraction would erase (Sect. 2.2). An explanation can describe a divergence only after some ground-truth representation has made it observable. Glassbox debugging supplies that representation, on top of which an abstraction-matched explanation can then be layered.

Read-only source visibility. The textual interface is exposed but not editable. This one view exposes information relevant to three transitional use cases: inspection of generated code without authoring it, comparison between visual constructs and textual syntax, and source-level inspection while preserving the visual workspace as the source-of-truth.

Cross-target execution. By running the same BLOCKLY program in several target languages and comparing outcomes side by side (Sect. 3.1), GLANCER surfaces language-specific behaviors that single-target debuggers cannot show. The cases of Sect. 5.2 show that this exposure makes the causes of otherwise silent failures observable.

Handoff to text-based debuggers. Breakpoint export through the BREAKPOINTIO convention closes the loop. When further investigation requires the depth of a whitebox tool, the exported metadata can be carried into the IDE without manual reconstruction. The transition becomes a graceful degradation of abstraction rather than a context switch.

5.4 Tradeoffs, Limitations, and Validity

The glassbox approach trades simplicity for transparency. Because GLANCER is a proof-of-concept implementation of the approach, our evaluation establishes diagnostic observability rather than user-level debugging effectiveness. Several tradeoffs and validity limits follow from this scope.

Performance overhead. The BVD4B-derived step-execution mechanism instruments generator output with asynchronous *Promises* so that execution can pause without altering program logic. This adds non-trivial overhead, especially for compute-heavy programs, and can perturb the very behavior the developer is trying to diagnose, an instance of the well-known debugging-interference problem [32].

Visual real estate. A glassbox session presents the visual workspace, several read-only code views, and a debugger

panel side by side. The layout fits an external monitor comfortably but is cramped on a laptop screen, especially with multiple target languages selected. Per-language toggling mitigates but does not eliminate this cost.

Generator opacity. GLANCER treats the generators themselves as part of the debuggable surface, but it does not help fix a generator. When a divergence has its root cause in a generator template rather than in language semantics, the developer can localize the divergence but must still touch the generator source to repair it.

Breakpoint policy. The policy, mentioned in Sect. 4.4, chooses consistency over fine-grained control. It cannot represent “break only on this leaf, not on its siblings” without breaking visual-textual symmetry. The choice is deliberate but constrains advanced users.

Single-platform binding. GLANCER is built on BLOCKLY. The block-to-code mapping algorithm generalizes to any platform with the same hierarchical block-to-horizontal-group-to-source-line correspondence, but porting the system to Scratch or other environments is non-trivial engineering.

Threats to validity. Debugging effectiveness and program understanding are difficult to quantify directly. Our evaluation does not attempt to measure them empirically and reports no controlled user study, so claims about supporting the transitional user rest on design-goal and diagnostic-coverage arguments rather than on behavioral evidence. Internal validity is shaped by the prototype maturity of GLANCER and by implementation choices, in particular the use of BLOCKLY as the visual host, the choice of JavaScript and Python as the primary execution targets, and a dependence on BLOCKLY’s per-language generators whose evolution may shift which divergences the system surfaces. External validity is limited by case-study scale: the diagnostic-coverage suite is a curated set of programs that exercise the visual-textual boundary rather than a representative sample of real-world failures. Finally, the instrumentation used to control execution flow introduces performance overhead that may affect the very behavior under investigation.

5.5 Directions for Future Work

Several topics are left for future work.

Generative AI as a divergence explainer. GLANCER’s cross-target execution exposes when the same visual program behaves differently across targets, but it does not explain why. The most natural extension is to feed the divergence and the surrounding generated source to a *Large Language Model (LLM)* prompted as a divergence explainer, so that the user receives a localized, language-grounded explanation alongside the side-by-side outputs. This is a narrowly-scoped use of *Generative AI (GAI)* that fits the glassbox loop specifically and depends on the loop’s existence: prior work

on integrating GAI with BLOCKLY [13] targeted authoring and faces the abstraction gap that Liu et al. [28] characterize, but our context inverts the gap: the LLM is interpreting concrete runtime artifacts, generated source with differing outputs, not generating new ones from intent.

Collaborative debugging. The snapshot and execution-log mechanism is structured for sharing. Building on prior work on collaborative BLOCKLY workspaces [48], layering a VCS-aware comparison view on top of GLANCER’s snapshots would let teams diff *debug sessions* the same way they diff source.

Generator-level debugging. Generator opacity is a structural limit of the current design. A future version could expose the generator templates themselves as a third synchronized interface, allowing the user to navigate from a divergent runtime behavior, to the generated source that produced it, to the generator template that emitted that source.

Integrating established debuggers. GLANCER instruments generated code from scratch rather than driving an existing language debugger. Building instead on a native debugger’s API, in the spirit of browser-based tools such as Python Tutor, could inherit performance work such as Python’s low-overhead monitoring interface (PEP 669) and lessen the instrumentation overhead noted in Sect. 5.4. For JavaScript targets, the same path could add time-travel debugging, letting the user step backward as well as forward through execution.

Portable debug metadata. The block-to-code mapping is currently specific to BLOCKLY’s generators. A standard debug-metadata format for block-language compilers, analogous to the debug symbols that let gdb serve both C and Rust, would let a single glassbox debugger attach to programs produced by different compilers, directly relaxing the single-platform binding noted in Sect. 5.4.

6 Related Work

Related work includes hybrid visual and textual programming, visual debugging platforms, source-level debugging across translations, and no-code generative AI integration.

6.1 Hybrid Visual and Textual Programming

A line of work has explored unifying visual and textual programming representations rather than treating them as separate paradigms. Erwig and Meyer [11] introduced *heterogeneous visual languages* that integrate visual and textual constructs into a single program. Homer and Noble [20] synthesize lessons from two decades of combining block-based and textual programming, identifying view synchronization and incremental disclosure of source as recurring design

themes. Beckmann et al. [2] propose cross-language domain-specific visual replacements within structured editors, showing how visual notation and textual notation can coexist in a single editing environment.

Prior research concentrates on hybrid *authoring*. GLANCER carries the same philosophy into *debugging*, where synchronization is between visual blocks and the multiple target-language sources their generators emit during execution rather than between two co-equal authoring views.

GLANCER presents synchronized representations of a single program artifact, similar to *projectional editors* [50] and *language workbenches* [10]. A key difference is that projectional editors usually support multi-view authoring, while GLANCER treats generated code as a read-only debugging view and synchronizes navigation and debugging actions, not arbitrary edits.

6.2 Visual Debugging Platforms

BVD4B [44] for BLOCKLY and NUZZLEBUG [8] for Scratch [31] represent state-of-the-art blackbox visual debuggers. Both expose flow control and state at the visual interface; neither exposes the generated code. WHYLINE [25] adds a why-question interface on top of program output, but stays at the visual interface as well. Empirical studies of how novices use such tools (Kim et al. [24], Hassan et al. [18]) report a cognitive cost when developers must switch between block-level debugging and source-level inspection, motivating synchronized support across block-level debugging and source-level inspection, which GLANCER provides. On top of unifying the two interfaces, GLANCER extends this workflow with VCS-style snapshots and cross-target validation, capabilities not provided by the visual debuggers discussed here.

6.3 Source-Level Debugging Across Translations

The challenge of debugging code that does not look like the code the developer wrote is not unique to no-code platforms. Kell and Stinnett [23] examine source-level debugging of compiler-optimized code, arguing that residual-state metadata is what keeps optimized binaries fully source-level debuggable. Marra et al. [32] propose *out-of-place debugging*, an architecture that decouples the debugged process from the debugger to reduce debugging interference. GLANCER shares the same impulse for no-code environments. The generated source is treated as a faithful but secondary view that must be reconciled to the visual program rather than abstracted away. The block-to-code mapping (Sect. 4) plays the role that debug metadata plays in optimized-binary debugging.

Like source maps and debug metadata, GLANCER preserves debuggability across transformations. However, GLANCER addresses the representation gap for visual programs rather than textual files.

6.4 No-Code Generative AI Integration

LLMs and GAI tools, including CHATGPT [38] and Google's GEMINI [47], have been integrated with no-code platforms to support code generation, with Too [1] and the CHATGPT-BLOCKLY integration for end-user robot programming [13] as representative examples. These efforts target program *authoring* and so are orthogonal to the abstraction-leak problem GLANCER addresses, but they raise a complementary need. As generated programs become more prevalent, debugging them across target-language semantics becomes more important, not less.

7 Conclusion

As software development evolves, no-code platforms, alongside generative AI tools, are key enablers in democratizing application creation for non-experts. However, the complexities inherent in the behavior of no-code software can at times be confusing even for experienced developers, let alone for citizen developers, often necessitating source-level debugging.

On the one hand, traditional whitebox debuggers excel at source-level debugging but lack the necessary support for debugging no-code visual components. On the other hand, blackbox approaches in existing no-code platforms provide visual debugging but conceal the underlying code from the debugger. This limitation is especially consequential for industrial no-code applications, where robust debugging capabilities are important for maintaining and troubleshooting complex systems [16].

We present a glassbox approach that utilizes the underlying generated code for debugging, leveraging the platform's ability to generate code in various languages and the potential availability of multiple debugging sessions and logs for each execution.

To concretely illustrate the approach, we present GLANCER, a debugger that implements the glassbox approach for BLOCKLY, coupled with cross-target and cross-session utilization. GLANCER introduces a synchronized debugging framework that connects visual and textual representations, allowing users to track execution across both interfaces. GLANCER provides not only source-level visibility into generated code, but also source-level debugging over generated code. It also provides cross-target validation, generating code in multiple languages to expose correctness-relevant divergences and help developers reason about correctness. On selected test programs, GLANCER exposed source-level behavior, code-comprehension cues, and cross-target validation evidence that were unavailable in the blackbox baseline.

While promising, GLANCER is not without limitations. Synchronizing views and execution can slow performance for larger programs, requiring optimization to maintain responsiveness. The handling of complex applications risks interface clutter, which requires scalable design solutions. The multi-language output, though powerful, can overwhelm

novices. Integrating with existing IDEs can also encounter compatibility issues, demanding broader ecosystem alignment. These challenges are topics left for future work and for future versions of GLANCER.

Our work establishes a foundation for more transparent and powerful no-code development platforms, empowering a broader range of citizen developers to engage in debugging their no-code programs.

References

- [1] Assaf Avishahar-Zeira and David H. Lorenz. 2023. Could No-Code Be Code? Toward a No-Code Programming Language for Citizen Developers. In *Proceedings of the 2023 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software* (Cascais, Portugal) (Onward! 2023). ACM, New York, NY, USA, 106–122. doi:10.1145/3622758.3622893
- [2] Tom Beckmann, Daniel Stachnik, Jens Lincke, and Robert Hirschfeld. 2023. Visual Replacements: Cross-Language Domain-Specific Representations in Structured Editors. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Programming Abstractions and Interactive Notations, Tools, and Environments* (Cascais, Portugal) (PAINT 2023). ACM, New York, NY, USA, 25–35. doi:10.1145/3623504.3623569
- [3] Andrea Bellucci, Paloma Díaz, and Ignacio Aedo. 2023. Exploring Visual Languages for Prototyping Interactive Behaviors for Tangible Virtual Reality. In *End-User Development: 9th International Symposium, IS-EUD 2023* (Bari, Italy) (Lecture Notes in Computer Science, Vol. 13917). Springer Nature, Cham, Switzerland, 203–219. doi:10.1007/978-3-031-34433-6_13
- [4] Björn Binzer, Edona Elshan, Daniel Fürstenau, and Till J. Winkler. 2024. Establishing a Low-Code/No-Code-Enabled Citizen Development Strategy. *MIS Quarterly Executive* 23, 3 (Sept. 2024), 253–273. doi:10.17705/2msqe.00097
- [5] Gilad Bracha. 2015. *The Dart programming language*. Addison-Wesley Professional, Boston, MA, USA.
- [6] Daurnimator and Benoit Giannangeli. 2017. Fengari: Lua in the Browser. Lua Workshop 2017 (LW 2017), San Francisco, CA, USA. <https://www.lua.org/wshop17/fengari.html>
- [7] Deckerio. 2019. BreakpointIO. <https://marketplace.visualstudio.com/items?itemName=deckerio.breakpointio>
- [8] Adina Deiner and Gordon Fraser. 2024. NuzzleBug: Debugging Block-based Programs in Scratch. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering* (Lisbon, Portugal) (ICSE 2024). IEEE, New York, NY, USA, 1–13. doi:10.1145/3597503.3623331 (arXiv:2309.14465).
- [9] Edona Elshan, Björn Binzer, and Till J. Winkler. 2024. From Software Users to Software Creators: An Exploration of the Core Characteristics of the Citizen Developer Role and the Related Re- and Upskilling Programs. *Business & Information Systems Engineering* 67 (Dec. 2024), 31–53. doi:10.1007/s12599-024-00915-1
- [10] Sebastian Erdweg, Tijs van der Storm, Markus Völter, Meinte Boersma, Remi Bosman, William R. Cook, Albert Gerritsen, Angelo Hulshout, Steven Kelly, Alex Loh, Gabriël D. P. Konat, Pedro J. Molina, Martin Palatnik, Risto Pohjonen, Eugen Schindler, Klemens Schindler, Riccardo Solmi, Vlad A. Vergu, Eelco Visser, Kevin van der Vlist, Guido H. Wachsmuth, and Jimi van der Woning. 2013. The State of the Art in Language Workbenches. In *Software Language Engineering*. Springer, Cham, Switzerland, 197–217. doi:10.1007/978-3-319-02654-1_11
- [11] Martin Erwig and Bernd Meyer. 1995. Heterogeneous Visual Languages: Integrating Visual and Textual Programming. In *Proceedings of the 11th IEEE International Symposium on Visual Languages* (Darmstadt, Germany) (VL '95). IEEE Computer Society, Los Alamitos, CA, USA, 318–325. doi:10.1109/VL.1995.520825

- [12] David Flanagan. 2011. *JavaScript: The Definitive Guide* (6th ed.). O'Reilly Media, Inc., Sebastopol, CA.
- [13] Luigi Gargioni and Daniela Fogli. 2024. Integrating ChatGPT with Blockly for End-User Development of Robot Tasks. In *Companion of the 2024 ACM/IEEE 19th International Conference on Human-Robot Interaction* (Boulder, Colorado, USA) (*HRI 2024*). ACM, New York, NY, USA, 478–482. doi:10.1145/3610978.3640653 (YouTube:8YnMJVrGQY4).
- [14] Google. 2012. Blockly: A Visual Programming Editor. <https://developers.google.com/blockly>.
- [15] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and Jean-François Bastien. 2017. Bringing the Web up to Speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Barcelona, Spain) (*PLDI 2017*). ACM, New York, NY, USA, 185–200. doi:10.1145/3062341.3062363
- [16] Brent Hailpern and Padmanabhan Santhanam. 2002. Software debugging, testing, and verification. *IBM Systems Journal* 41, 1 (2002), 4–12. doi:10.1147/sj.411.0004
- [17] Handsontable. 2023. JavaScript Data Grid with Spreadsheet UI. <https://handsontable.com/>
- [18] Mohammed Hassan, Grace Zeng, and Craig Zilles. 2024. Evaluating How Novices Utilize Debuggers and Code Execution to Understand Code. In *Proceedings of the 2024 ACM Conference on International Computing Education Research* (Melbourne, VIC, Australia) (*ICER 2024, Vol. 1*). ACM, New York, NY, USA, 65–83. doi:10.1145/3632620.3671126
- [19] Marijn Haverbeke and CodeMirror Contributors. 2025. CodeMirror: Extensible Code Editor. <https://codemirror.net/>.
- [20] Michael Homer and James Noble. 2017. Lessons in Combining Block-based and Textual Programming. *Journal of Visual Languages and Sentient Systems* 3, 1 (July 2017), 22–39. doi:10.18293/VLSS2017-007
- [21] Roberto Ierusalimschy. 2006. *Programming in Lua*. Roberto Ierusalimschy, Departamento de Informática, PUC-Rio, Rio de Janeiro, RJ, Brazil.
- [22] Sebastian Brustad Kaland, Scott Aleksander Bekke, and Marcus Balcan. 2023. *A study on no-code technology as a resource for competitive advantage*. Master's thesis. Dept. of Industrial Economics and Technology Management, Faculty of Economics and Management, Norwegian University of Science and Technology (NTNU). <https://hdl.handle.net/11250/3117094>
- [23] Stephen Kell and J. Ryan Stinnett. 2024. Source-Level Debugging of Compiler-Optimised Code: Ill-Posed, but Not Impossible. In *Proceedings of the 2024 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (Pasadena, CA, USA) (*Onward! '24*). ACM, New York, NY, USA, 38–53. doi:10.1145/3689492.3690047
- [24] ChanMin Kim, Jiangmei Yuan, Lucas Vasconcelos, Minyoung Shin, and Roger B Hill. 2018. Debugging during block-based programming. *Instructional Science* 46 (2018), 767–787. doi:10.1007/s11251-018-9453-5
- [25] Amy J. Ko and Brand Myers. 2004. Designing the Whyline: A Debugging Interface for Asking Questions about Program Behaviour. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Vienna Austria) (*CHI 2004*). ACM, New York, NY, USA, 151–158. doi:10.1145/985692.985712
- [26] Glenn E. Krasner and Stephen T. Pope. 1988. A Cookbook for Using the Model-View Controller User Interface Paradigm in Smalltalk-80. *Journal of Object Oriented Programming* 1, 3 (August/September 1988), 26–49.
- [27] Rasmus Jay Lerdorf, Kevin Tatro, Bob Kaehms, and Ric McGredy. 2002. *Programming PHP*. O'Reilly & Associates, Inc., Sebastopol, CA, USA.
- [28] Michael Xieyang Liu, Advait Sarkar, Carina Negreanu, Benjamin Zorn, Jack Williams, Neil Toronto, and Andrew D. Gordon. 2023. "What it wants me to say": Bridging the abstraction gap between end-user programmers and code-generating large language models. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) (*CHI '23*). ACM, New York, NY, USA, 1–31. doi:10.1145/3544548.3580817
- [29] Jon Loeliger and Matthew McCullough. 2012. *Version Control with Git: Powerful tools and techniques for collaborative software development* (2nd ed.). O'Reilly Media, Inc., Sebastopol, CA, USA.
- [30] Mark Lutz. 1996. *Programming Python* (1st ed.). O'Reilly, Sebastopol, CA.
- [31] David J. Malan and Henry H. Leitner. 2007. Scratch for Budding Computer Scientists. *ACM SIGCSE Bulletin* 39, 1 (March 2007), 223–227. doi:10.1145/1227504.1227388
- [32] Matteo Marra, Guillermo Polito, and Elisa Gonzalez Boix. 2019. Out-Of-Place Debugging: A Debugging Architecture to Reduce Debugging Interference. *The Art, Science, and Engineering of Programming* 3, 2, Article 3 (2019), 34 pages. doi:10.22152/programming-journal.org/2019/3/3
- [33] Charles E. McDowell and David P. Helmbold. 1989. Debugging Concurrent Programs. *ACM Computing Surveys (CSUR)* 21, 4 (Dec. 1989), 593–622. doi:10.1145/76894.76897
- [34] Tilman Michaeli, Stefan Seegerer, and Ralf Romeike. 2018. Enabling Collaboration and Tinkering: A Version Control System for Block-based Languages. In *Proceedings of the Conference on Constructionism, Computational Thinking and Educational Innovation* (Vilnius, Lithuania) (*Constructionism 2018*), Valentina Dagienė and Eglė Jasutė (Eds.). Vilnius University, Vilnius, Lithuania, 405–414. <http://www.constructionism2018.fsf.vu.lt/proceedings>
- [35] Microsoft. 2015. Visual Studio Code. <https://code.visualstudio.com/>
- [36] Siti Nor Hafizah Mohamad, Ahmed Patel, Rodziah Latih, Qais Qassim, Liu Na, and Yiqi Tew. 2011. Block-based Programming Approach: Challenges and Benefits. In *Proceedings of the 2011 International Conference on Electrical Engineering and Informatics*. IEEE, NJ, USA, 1–5. doi:10.1109/ICEEI.2011.6021507
- [37] Sean Morris. 2024. php-wasm: PHP Compiled to WebAssembly. <https://github.com/seanmorris/php-wasm>
- [38] OpenAI. 2023. ChatGPT: Optimizing Language Models for Dialogue. <https://www.openai.com/chatgpt>.
- [39] Daniel Parker. 2015. *JavaScript with Promises: Managing Asynchronous Code*. "O'Reilly Media, Inc.", Sebastopol, CA.
- [40] Pyodide Contributors. 2025. Pyodide: Python Distribution for the Browser and Node.js Based on WebAssembly. <https://pyodide.org/>
- [41] Python Software Foundation. 2024. *pdb - The Python Debugger*. Python Software Foundation. <https://docs.python.org/3/library/pdb.html> Python Standard Library documentation.
- [42] Alexander Repenning. 2017. Moving Beyond Syntax: Lessons from 20 Years of Blocks Programming in AgentSheets. *Journal of Visual Languages and Sentient Systems* 3 (July 2017), 68–91. doi:10.18293/VLSS2017-010
- [43] Advait Sarkar. 2023. Will Code Remain a Relevant User Interface for End-User Programming with Generative AI Models?. In *Proceedings of the 2023 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (*Onward! 2023*). ACM, New York, NY, USA, 153–167. doi:10.1145/3622758.3622882
- [44] Anthony Savidis and Crystalia Savaki. 2019. Complete Block-Level Visual Debugger for Blockly. In *Proceedings of the 2nd International Conference on Human Systems Engineering and Design: Future Trends and Applications* (Universität der Bundeswehr München, Munich, Germany) (*IHSED 2019*). Springer Nature Switzerland, Switzerland, 286–292. doi:10.1007/978-3-030-27928-8_43
- [45] Richard Stallman, Roland Pesch, Stan Shebs, et al. 1988. *Debugging with GDB: The GNU Source-Level Debugger*. Free Software Foundation, Boston, MA, USA. <https://www.gnu.org/software/gdb/documentation/>

- [46] Giancarlo Succi, Andrea Valerio, Tullio Vernazza, and Gianpiero Succi. 1998. Compatibility, Standards, and Software Production. *ACM Stand.* 6, 4 (1998), 140–146. doi:[10.1145/338183.338185](https://doi.org/10.1145/338183.338185)
- [47] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: A Family of Highly Capable Multimodal Models. *arXiv preprint abs/2312.11805* (2023), 1–62. arXiv:[2312.11805](https://arxiv.org/abs/2312.11805) <https://arxiv.org/abs/2312.11805>
- [48] Yannis Valsamakis, Anthony Savidis, Emanuel Agapakis, and Alexandros Katsarakis. 2020. Collaborative visual programming workspace for Blockly. In *2020 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, Dunedin, New Zealand, 1–6. doi:[10.1109/VL/HCC50065.2020.9127253](https://doi.org/10.1109/VL/HCC50065.2020.9127253)
- [49] Altus Viljoen, Benedikt Stelzl, Maoying Yang, John Nguyen, Andreas Hein, Edona Elshan, and Helmut Krcmar. 2026. Navigating Flexibility and Standardisation in Low-Code/No-Code Development. *Information Systems Journal* 36, 1 (2026), 95–109. doi:[10.1111/isj.70001](https://doi.org/10.1111/isj.70001)
- [50] Markus Völter, Tamás Szabó, Sascha Lisson, Bernd Kolb, Sebastian Erdweg, and Thorsten Berger. 2016. Efficient Development of Consistent Projectional Editors Using Grammar Cells. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Software Language Engineering*. ACM, New York, NY, USA, 28–40. doi:[10.1145/2997364.2997365](https://doi.org/10.1145/2997364.2997365)